

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

hyh

xiaoming90

Date Audited: **July 23 - July 24, 2026**

Introduction

BlueBuyCallback is a periphery contract that lets an owner fund Midnight buy offers with liquidity supplied through a Morpho Blue position held by the callback. During settlement, it withdraws the required loan tokens from the configured Blue market and approves Midnight to collect them.

Scope

Repository: [morpho-org/midnight](https://github.com/morpho-org/midnight)

Audited Commit: [f826f2379697fc7432b862083798ad6432601f8a](#)

Final Commit: [eba62f704e58acde9a77f341f49f52f18f3900b3](#)

Files:

- [src/periphery/BlueBuyCallbackFactory.sol](#)
- [src/periphery/BlueBuyCallback.sol](#)

Final Commit Hash

[eba62f704e58acde9a77f341f49f52f18f3900b3](#)

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d6166

A handwritten signature in white ink on a black background. The signature consists of a large, stylized capital 'S' that loops around the word 'Simao', which is written in a cursive script.

@hyh

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'hyh'.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'Xiaomingao'.

Issue L-1: BlueBuyCallback balance is unreachable for owner [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-midnight-update-july-23rd-2026/issues/3>

Summary

Anything sent to the callback is permanently locked as BlueBuyCallback has no skim or leftover balance capturing logic, while that's an address users can read as their escrow, and the factory publishes it via `callback0f`.

Impact

Any tokens sent to callback directly as an operational mistake or rewards received by a callback during Blue and Midnight operations are locked with the contract forever.

Recommendation

Consider adding an `onlyOwner` skim functionality or documenting the limitation.

Discussion

MathisGD

Fixed by 1096, to be able to get back Morpho Blue rewards

Issue L-2: `buyerAssetsBound()` doesn't replicate `onBuy()` restrictions fully [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-midnight-update-july-23rd-2026/issues/4>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`buyerAssetsBound()` doesn't check the `loanToken` consistency that `onBuy()` does enforce, so a router can receive a nonzero bound for data that `onBuy()` will reject.

Impact

Routing layer is open to make an operational mistakes of querying the wrong market and failing the batches populated by unrealistic bounds.

Code Snippet

`buyerAssetsBound()` ignores the market and doesn't control for the loan token mismatch:

```
/// @dev Reverts if the owner position on the requested market is too small or
↪ if the liquidity on that market is
/// too small.
function onBuy(
    ...
) external returns (bytes32) {
    require(msg.sender == MIDNIGHT, NotMidnight());
    require(buyer == OWNER, NotOwnerBuyer());
    MarketParams memory marketParams = abi.decode(data, (MarketParams));
>>    require(marketParams.loanToken == market.loanToken,
↪ InconsistentLoanToken());
```

```

    if (buyerAssets > 0) IMorpho(BLUE).withdraw(marketParams, buyerAssets, 0,
        ↪ address(this), address(this));
    forceApproveMax(market.loanToken, MIDNIGHT);

    return CALLBACK_SUCCESS;
}

/// @dev Max buyerAssets amount that the callback can handle.
/// @dev This function is useful for bundles to query how much is available at
    ↪ the moment.
/// @dev Other buy callbacks might not take all constraints into account to
    ↪ provide their bound, but this is fine,
/// if the routing layer can take into account the other reasons.
>> function buyerAssetsBound(bytes32, Market memory, address, bytes memory data)
    ↪ external view returns (uint256) {
    MarketParams memory marketParams = abi.decode(data, (MarketParams));

    (uint256 totalSupplyAssets, uint256 totalSupplyShares, uint256
        ↪ totalBorrowAssets,) =
        IMorpho(BLUE).expectedMarketBalances(marketParams);
    uint256 supplyAssets = IMorpho(BLUE).position(marketParams.id(),
        ↪ address(this)).supplyShares
        .toAssetsDown(totalSupplyAssets, totalSupplyShares);
    uint256 liquidity = totalSupplyAssets - totalBorrowAssets;
    uint256 blueBalance = IERC20(marketParams.loanToken).balanceOf(BLUE);

    return UtilsLib.min(UtilsLib.min(supplyAssets, liquidity), blueBalance);
}

```

Recommendation

Consider replicating `onBuy()` enforced set of conditions in `buyerAssetsBound()`.

Discussion

MathisGD

We have acknowledged and explained this with a dev comment on the bound function

Issue L-3: Front-running createBlueBuyCallback reverts victims' bundled transactions via CREATE2 collision [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-midnight-update-july-23rd-2026/issues/5>

Vulnerability Detail

The CREATE2 will revert upon re-creation. In an extreme edge case where someone bundles a number of operations and one of them is the `createBlueBuyCallback` operation, a malicious user could front-run the `createBlueBuyCallback` and cause the "bundle" transaction to revert.

<https://github.com/sherlock-audit/2026-07-morpho-midnight-update-july-23rd-2026/blob/825a6ec0784674fec610f86a75c587c896381678/BlueBuyCallbackFactory.sol#L20>

```
function createBlueBuyCallback(address owner) external returns (address) {
    address callback = address(new BlueBuyCallback(salt: bytes32(0))(owner,
    ↪ MIDNIGHT, BLUE));
    callbackOf[owner] = callback;
    isBlueCallback[callback] = true;

    emit CreateBlueBuyCallback(owner, callback);
    return callback;
}
```

Impact

Attacker front-runs `createBlueBuyCallback(owner)`, causing the victim's identical CREATE2 deployment to revert, which breaks any bundle containing it.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-midnight-update-july-23rd-2026/blob/825a6ec0784674fec610f86a75c587c896381678/BlueBuyCallbackFactory.sol#L20>

Tool Used

Manual Review

Recommendation

Consider adding `if (callbackOf[owner] != address(0)) return callbackOf[owner];` to mitigate this minor griefing vector.

Discussion

MathisGD

This is fixed now, the factory allows no-ops

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.